R.Vishnu Vardhan - 2520090166

SKILL_WEEK-4

Task 1:

```
Last login: Thu Aug 13 19:56:29 on ttys007
root@Vishnu:~#                                cd ~/OSSP
pwd
/Users/Vishnu/OSSP
root@Vishnu:~#
```

Figure 1: OSSP project directory

```
  UW PICO 5.09                                File: src/parser.c

#include <stdio.h>
#include <string.h>

#define MAX_TOKENS 10
#define MAX_LENGTH 100

int main()
{
    char input[MAX_LENGTH];
    char *tokens[MAX_TOKENS];

    printf("my_shell> ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
    {
        printf("Empty command!\n");
        return 0;
    }

    int i = 0;

    tokens[i] = strtok(input, " ");

    while (tokens[i] != NULL && i < MAX_TOKENS - 1)
    {
        i++;
        tokens[i] = strtok(NULL, " ");
    }

    printf("\nTokens:\n");

    for (int j = 0; j < i; j++)
    {
        printf("Token %d = %s\n", j + 1, tokens[j]);
    }

    return 0;
}

^G Get Help  ^O WriteOut  ^R Read File  ^Y Prev Pg  ^K Cut Text  ^C Cur Pos
^X Exit      ^J Justify   ^W Where is   ^V Next Pg  ^U UnCut Text ^T To Spell
```

Figure 2: C program for parsing and lexical tokenization

```
root@Vishnu:~#                          gcc src/parser.c -o bin/parser
root@Vishnu:~#
```

Figure 3: Successful compilation of parser.c

Figure 4: Tokenization of the command ls -l docs

Figure 5: Tokenization of the command mkdir project

Figure 6: Detection and handling of an empty command

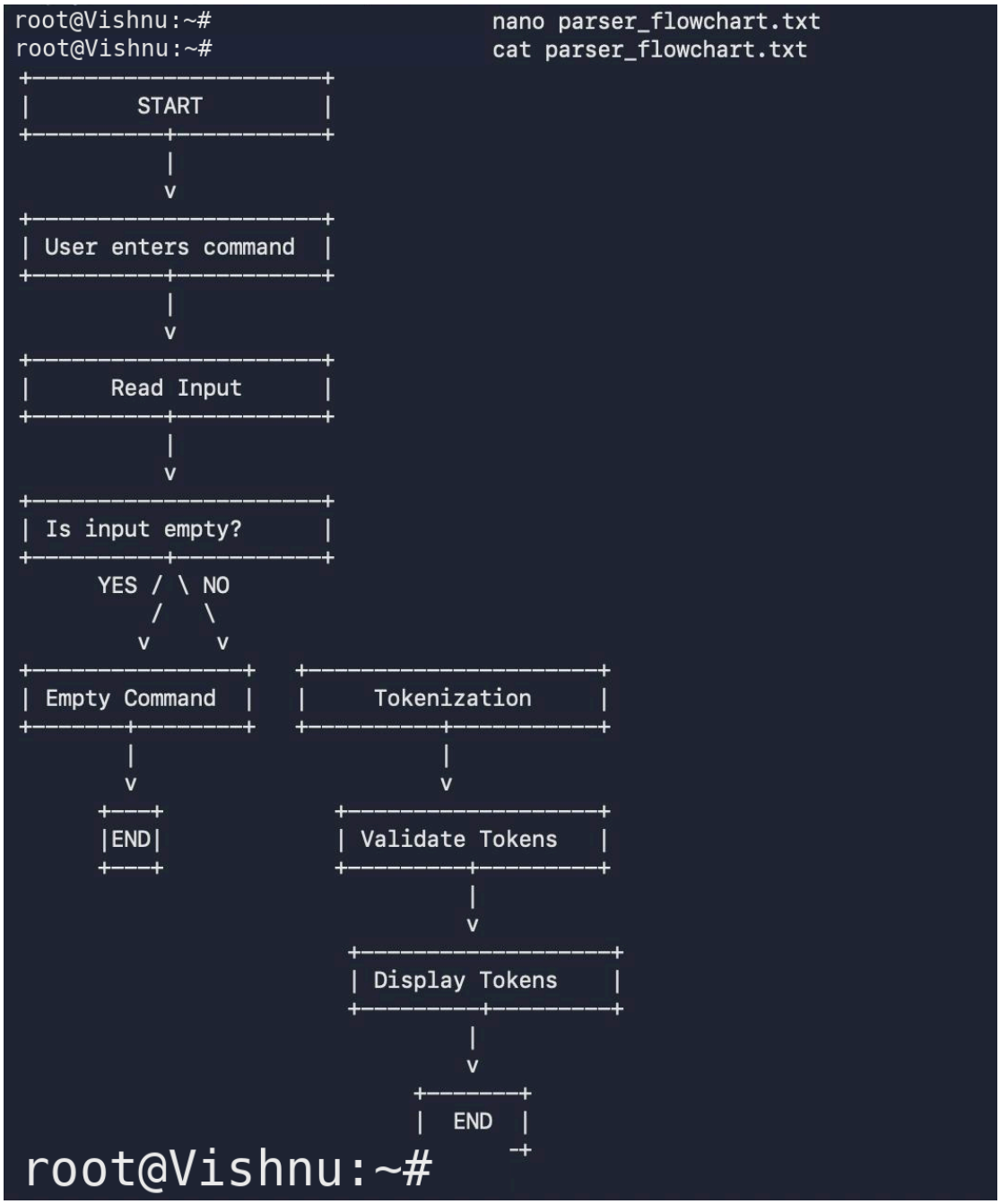


Figure 7: Flow chart for Parsing and Lexical Analysis

Task 2:

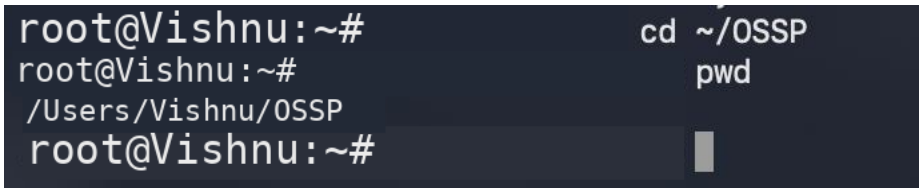


Figure 1: OSSP project directory for Task 2

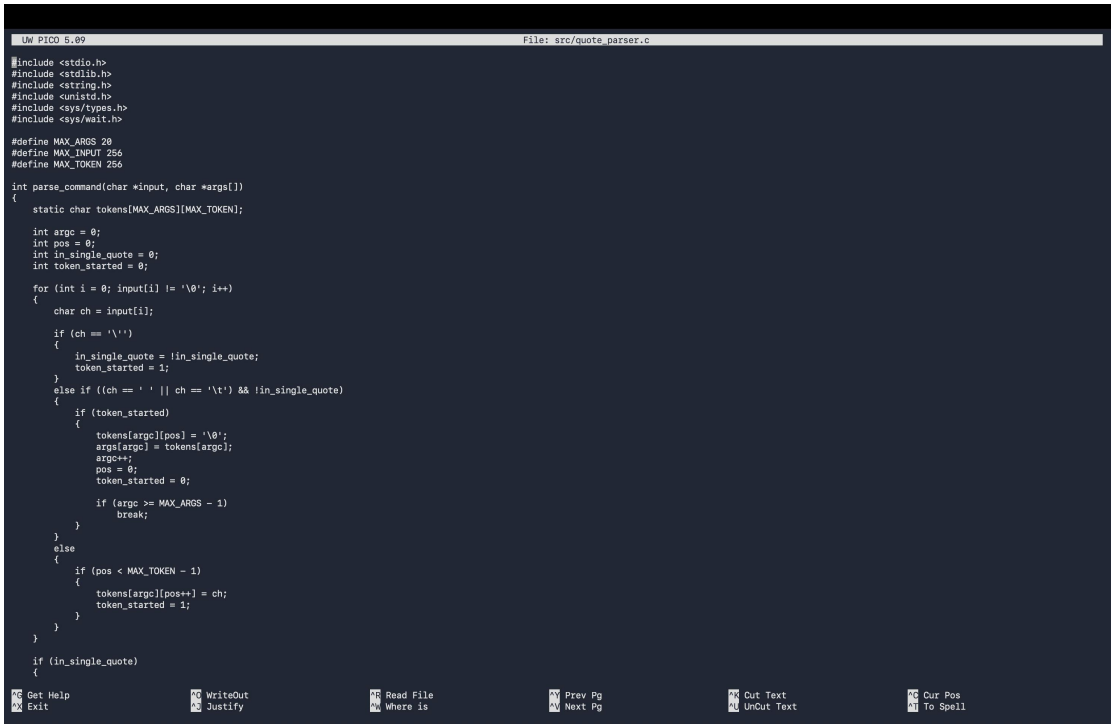


Figure 2: C program for single-quote parsing and command execution

Figure 3: Successful compilation of quote parser

Case 1:(a)

```
root@Vishnu:~#                    ./bin/quote_parser
myShell> echo Hello

Parsing Result:
Argument 1: echo
Argument 2: Hello

Child PID: 4395
Hello

Command execution completed.
root@Vishnu:~#
```

Figure 4: Parsing and execution of a normal command

Case 1:(b)

```
root@Vishnu:~#                    ./bin/quote_parser
myShell> echo 'Hello World'

Parsing Result:
Argument 1: echo
Argument 2: Hello World

Child PID: 4752
Hello World

Command execution completed.
root@Vishnu:~#
```

Figure 5: Single quotes preserving literal content as one argument

Case 2:

```
root@Vishnu:~#                    ./bin/quote_parser
myShell> echo 'Hello World
Syntax Error: Unmatched single quote.
root@Vishnu:~#
```

Figure 6: Detection of unmatched single quote and syntax error

```
root@Vishnu:~#                                ./bin/quote_parser
myShell> echo '$HOME'

Parsing Result:
Argument 1: echo
Argument 2: $HOME

Child PID: 5178
$HOME

Command execution completed.
root@Vishnu:~#
```

Figure 7: Single quotes preserving literal $HOME

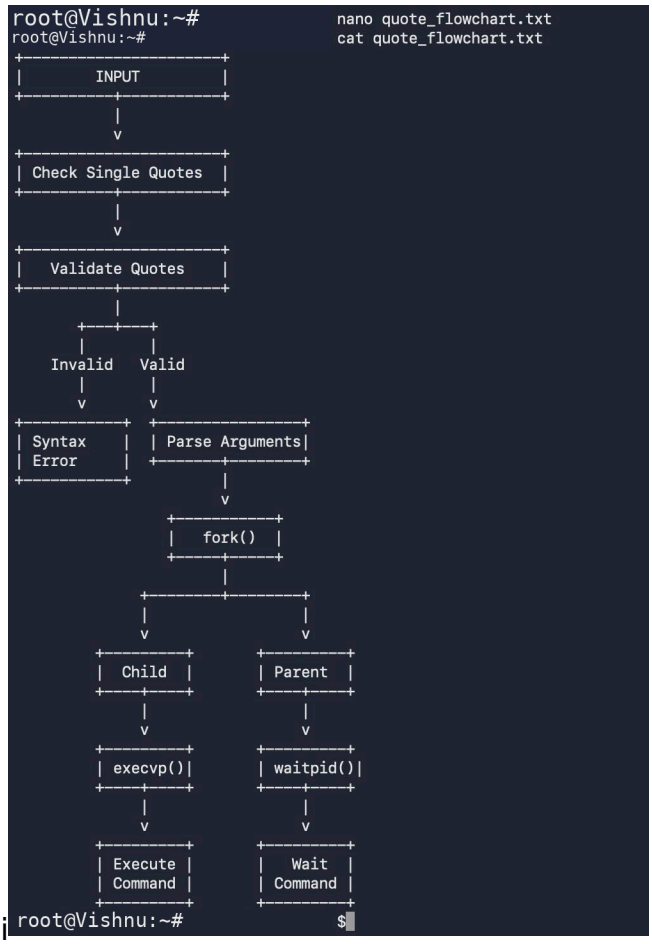


Figure 8: Terminal representation of the flowchart for single-quote parsing and command execution

Task 3:

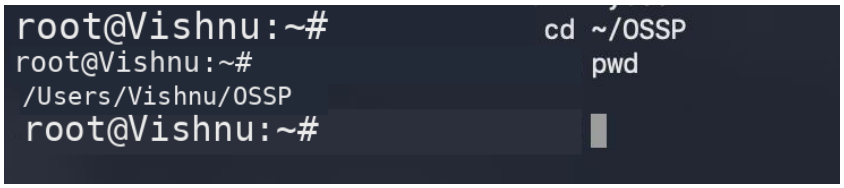
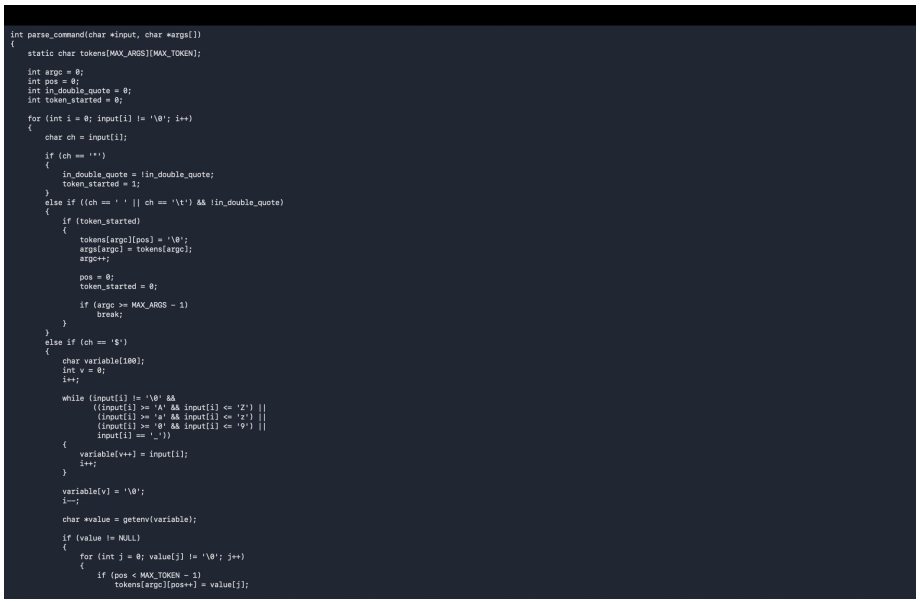
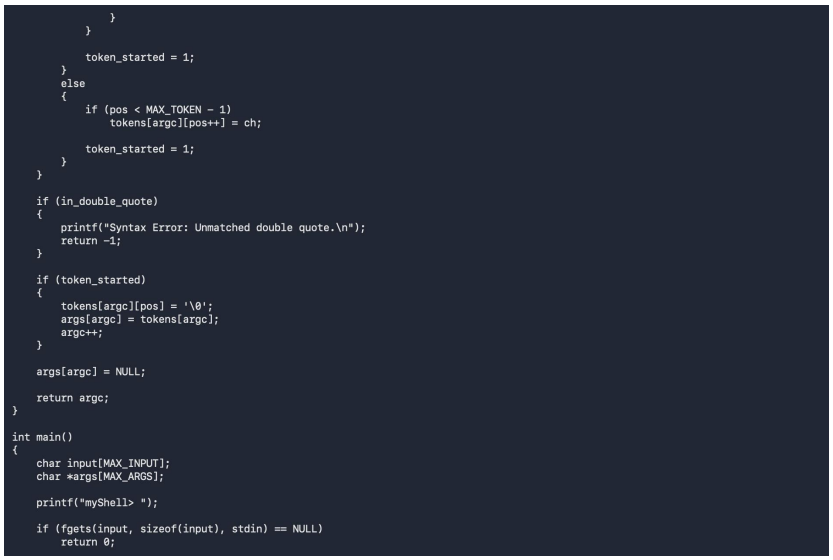


Figure 1: OSSP project directory for Task 4



```
int parse_command(char *input, char *args[])
{
    static char tokens[MAX_ARGS][MAX_TOKEN];

    int argc = 0;
    int pos = 0;
    int in_double_quote = 0;
    int token_started = 0;

    for (int i = 0; input[i] != '\0'; i++)
    {
        char ch = input[i];

        if (ch == '"')
        {
            in_double_quote = !in_double_quote;
            token_started = 1;
        }
        else if ((ch == ' ' || ch == '\t') && !in_double_quote)
        {
            if (token_started)
            {
                tokens[argc][pos] = '\0';
                args[argc] = tokens[argc];
                argc++;

                pos = 0;
                token_started = 0;

                if (argc >= MAX_ARGS - 1)
                    break;
            }
        }
        else if (ch == '$')
        {
            char variable[100];
            int v = 0;
            i++;

            while (input[i] != '\0' &&
                   ((input[i] >= 'A' && input[i] <= 'Z') ||
                    (input[i] >= 'a' && input[i] <= 'z') ||
                    (input[i] >= '0' && input[i] <= '9') ||
                    input[i] == '_'))
            {
                variable[v++] = input[i];
                i++;
            }

            variable[v] = '\0';
            i--;

            char *value = getenv(variable);

            if (value != NULL)
            {
                for (int j = 0; value[j] != '\0'; j++)
                {
                    if (pos < MAX_TOKEN - 1)
                        tokens[argc][pos++] = value[j];
```

Figure 2(a): Double-quote parsing program – input and quote validation



```
                }
            }

            token_started = 1;
        }
        else
        {
            if (pos < MAX_TOKEN - 1)
                tokens[argc][pos++] = ch;

            token_started = 1;
        }
    }

    if (in_double_quote)
    {
        printf("Syntax Error: Unmatched double quote.\n");
        return -1;
    }

    if (token_started)
    {
        tokens[argc][pos] = '\0';
        args[argc] = tokens[argc];
        argc++;
    }

    args[argc] = NULL;

    return argc;
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("myShell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 0;
```

Figure 2(b): Double-quote parsing program – variable expansion and tokenization

```
    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
    {
        printf("Empty command.\n");
        return 0;
    }

    int argc = parse_command(input, args);

    if (argc < 0)
        return 1;

    printf("\n--- Parsed Arguments ---\n");

    for (int i = 0; i < argc; i++)
    {
        printf("Argument %d: %s\n", i + 1, args[i]);
    }

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("\nChild Process\n");
        printf("PID = %d\n\n", getpid());

        execvp(args[0], args);

        perror("execvp");
        exit(1);
    }
    else
    {
        waitpid(pid, NULL, 0);
        printf("\nChild process completed.\n");
    }

    return 0;
}
root@Vishnu:~#
```

Figure 2(c): Double-quote parsing program – process creation and command execution

Case 1:

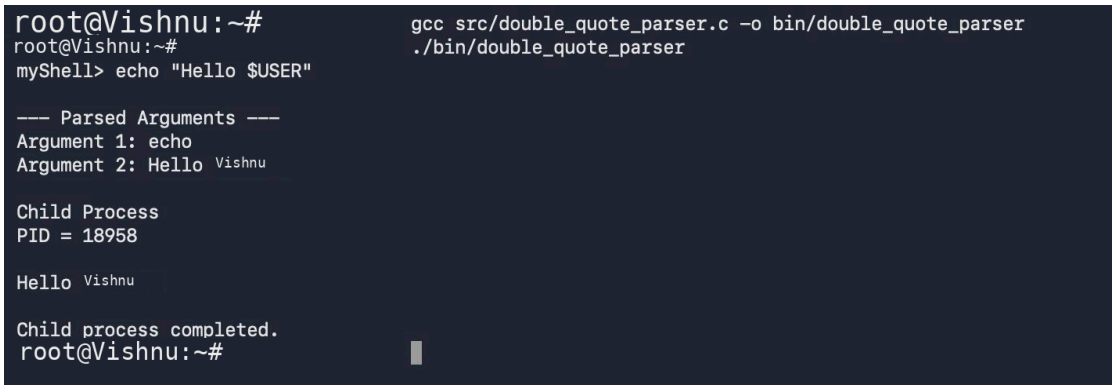
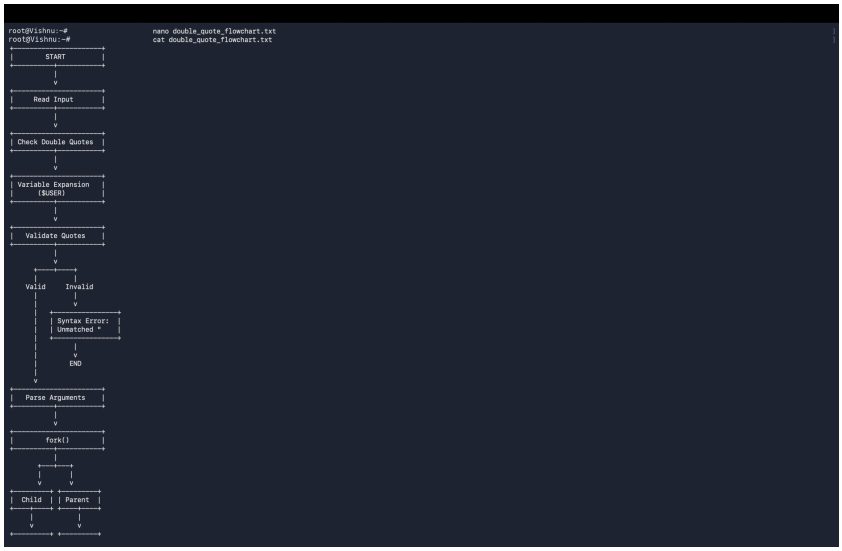


Figure 3: Successful compilation of double-quote parser and
Double quotes preserving spaces and expanding the $USER variable

Case 2:

Figure 4: Detection of unmatched double quote

Figure 5: Double quotes preserving spaces as a single argument



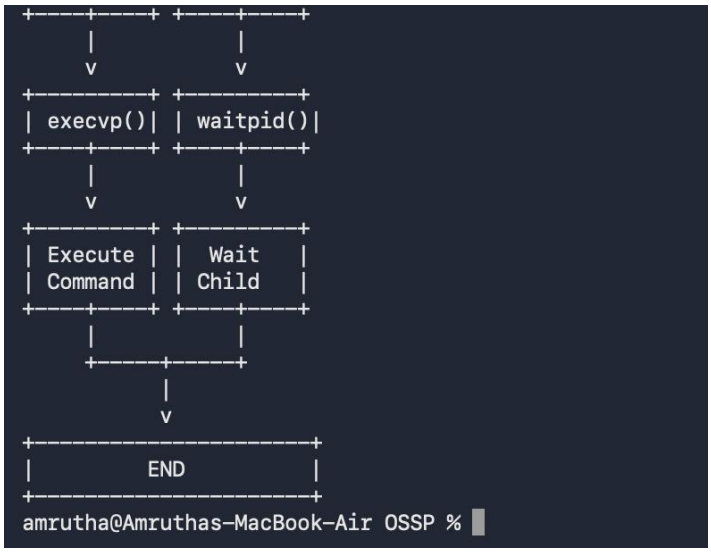


Figure 6: Terminal flowchart for double-quote validation, variable expansion and command execution

Figure 7: Git status after implementing double-quote parsing

Task 4:

```
root@Vishnu:~#                cd ~/OSSP
root@Vishnu:~#                pwd
/Users/Vishnu/OSSP
```

Figure 1: OSSP project directory for Task 4

```
int parse_command(char *input, char *args[])
{
    static char tokens[MAX_ARGS][MAX_TOKEN];

    int argc = 0;
    int pos = 0;
    int escaped = 0;
    int token_started = 0;

    for (int i = 0; input[i] != '\0'; i++)
    {
        char ch = input[i];

        if (escaped)
        {
            if (pos < MAX_TOKEN - 1)
                tokens[argc][pos++] = ch;

            escaped = 0;
            token_started = 1;
        }
        else if (ch == '\\')
        {
            escaped = 1;
            token_started = 1;
        }
        else if (ch == ' ' || ch == '\t')
        {
            if (token_started)
            {
                tokens[argc][pos] = '\0';
                args[argc] = tokens[argc];
                argc++;

                pos = 0;
                token_started = 0;

                if (argc >= MAX_ARGS - 1)
                    break;
            }
        }
        else
        {
            if (pos < MAX_TOKEN - 1)
                tokens[argc][pos++] = ch;

            token_started = 1;
        }
    }

    if (escaped)
    {
        printf("Syntax Error: Trailing escape character.\n");
        return -1;
    }

    if (token_started)
    {
        tokens[argc][pos] = '\0';
        args[argc] = tokens[argc];
```

Figure 2(a): Escape sequence parser – input processing and escape handling

```
    return argc;
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("myShell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 0;

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
    {
        printf("Empty command.\n");
        return 0;
    }

    int argc = parse_command(input, args);

    if (argc < 0)
        return 1;

    printf("\n--- Parsed Arguments ---\n");

    for (int i = 0; i < argc; i++)
    {
        printf("Argument %d: %s\n", i + 1, args[i]);
    }

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("\nChild Process\n");
        printf("PID = %d\n\n", getpid());

        execvp(args[0], args);

        perror("execvp");
        exit(1);
    }
    else
    {
        waitpid(pid, NULL, 0);
        printf("\nChild process completed.\n");
    }

    return 0;
}
root@Vishnu:~#
```

Figure 2(b): Escape sequence parser – tokenization and validation and process creation and command execution

Figure 3: Successful compilation of escape sequence parser

Case 1:

Figure 4: Case 1 – Handling escaped spaces as a single argument

Case 2:

Figure 5: Case 2 – Preserving an escaped special character

```
root@Vishnu:~#                    ./bin/escape_parser
myShell> echo My\ Project\ Folder

--- Parsed Arguments ---
Argument 1: echo
Argument 2: My Project Folder

Child Process
PID = 47541

My Project Folder

Child process completed.
root@Vishnu:~#
```

Figure 6: Parsing multiple escaped spaces in a complex command.

```
root@Vishnu:~#                    ./bin/escape_parser
myShell> echo Hello\
Syntax Error: Trailing escape character.
root@Vishnu:~#
```

Figure 7: Detection of an invalid trailing escape character

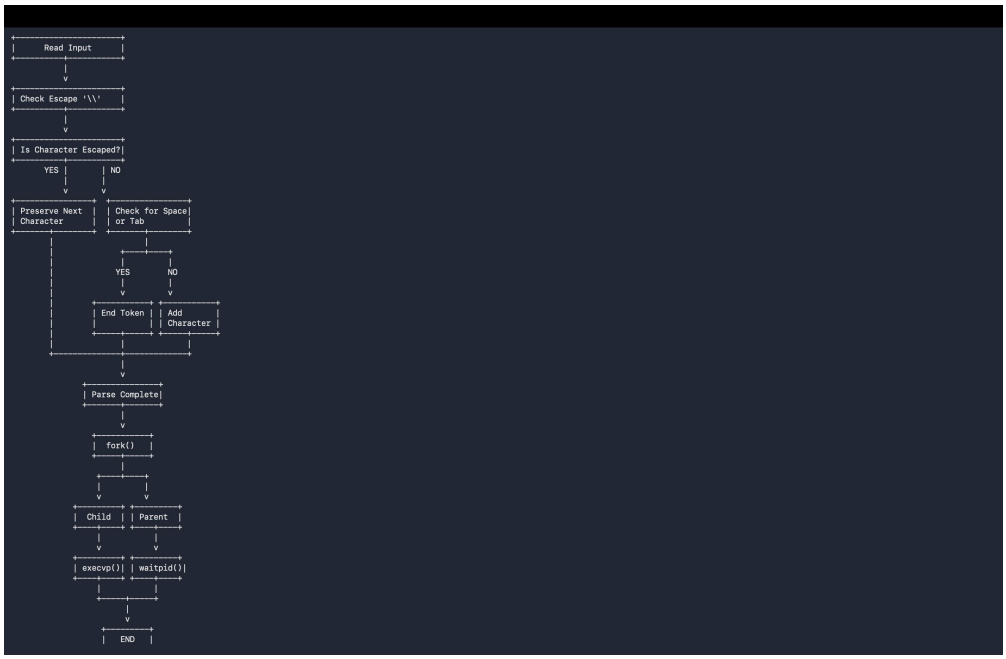


Figure 8: Terminal flowchart for escape sequence parsing and command execution

Task 5:

```
root@Vishnu:~#                    cd ~/OSSP
root@Vishnu:~#                    pwd
/Users/Vishnu/OSSP
root@Vishnu:~#
```

Figure 1: OSSP project directory.

```
root@Vishnu:~#                    cat src/process_launcher.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    char command[100];
    char argument[100];

    printf("myshell$ ");

    scanf("%99s", command);

    if (scanf("%99s", argument) == 1)
    {
        /* Argument is provided */
    }
    else
    {
        argument[0] = '\0';
    }

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }
```

Figure 2(a): Process launcher program using fork() and execvp()

```
    if (pid == 0)
    {
        printf("Child Process PID: %d\n", getpid());

        if (argument[0] != '\0')
        {
            char *args[] = {command, argument, NULL};
            execvp(command, args);
        }
        else
        {
            char *args[] = {command, NULL};
            execvp(command, args);
        }

        perror("Execution failed");
        exit(1);
    }
    else
    {
        printf("Parent Process PID: %d\n", getpid());
        printf("Created Child PID: %d\n", pid);

        int status;
        waitpid(pid, &status, 0);

        if (WIFEXITED(status))
        {
            printf("Child exited with status: %d\n",
                   WEXITSTATUS(status));
        }
    }

    return 0;
}
root@Vishnu:~#
```

Figure 2(b): Parent process management using waitpid() and exit-status handling

Case 1:

```
root@Vishnu:~#              ./bin/process_launcher
myshell$ ls
Parent Process PID: 91668
Created Child PID: 91687
Child Process PID: 91687
assets                  double_quote_flowchart.txt  File2.txt           Makefile              quote_flowchart.txt  src
bin                     escape_flowchart.txt        include             obj                   README.md            tests
docs                    File1.txt                   LICENSE             parser_flowchart.txt  scripts
Child exited with status: 0
root@Vishnu:~#
```

Figure 3: Case 1 – Successful execution of the ls command using a child process

Case 2:

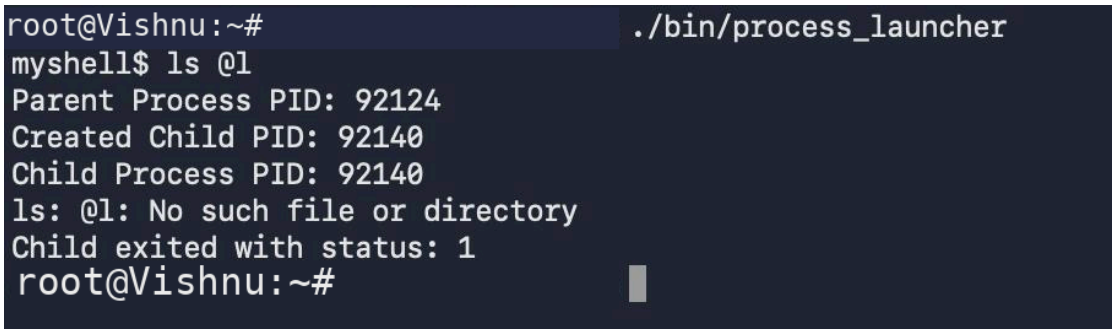


Figure 4: Case 2 – Handling an invalid argument and execution error

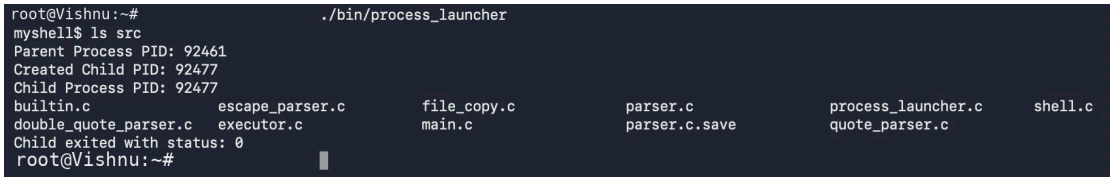


Figure 5: Passing an argument to the child process and executing the ls command

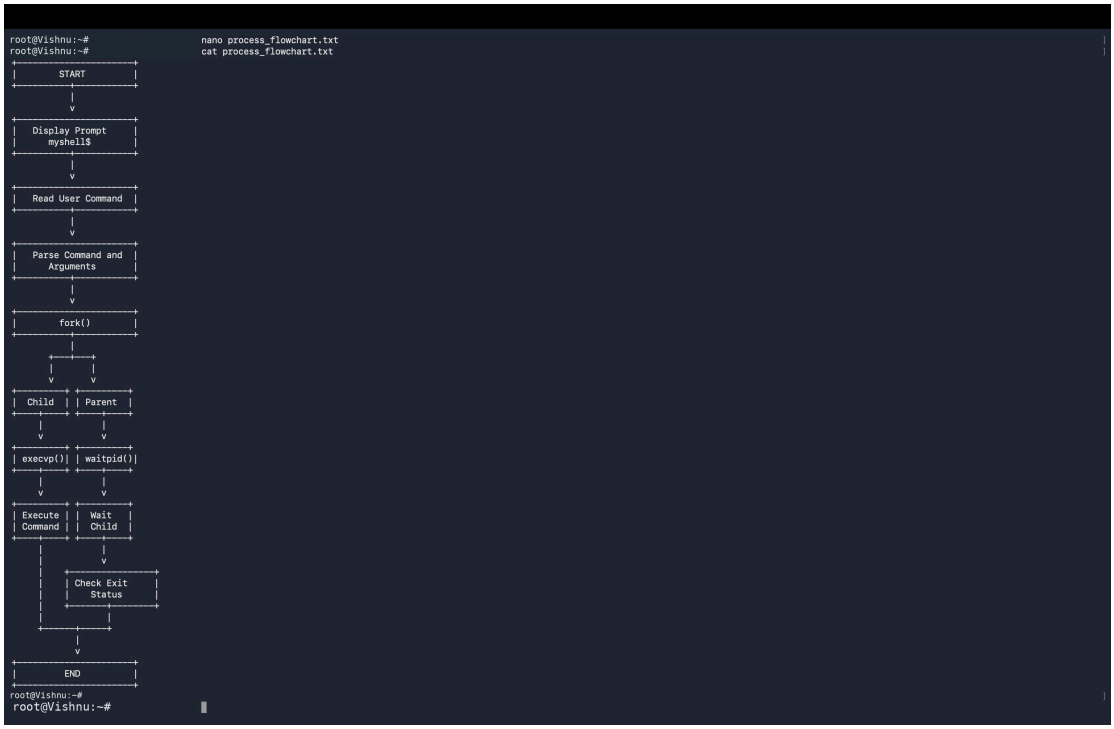


Figure 6: Terminal flowchart for process creation, command execution and parent process management